



Dryrun

Decoupling the Process

We take a look at Dryrun, an email-based interface via which you can run programs.

Dryrun (<https://bitbucket.org/amitsaha/dryrun>) attempts to present an email-based interface to run your programs. For example, if icanhazoutput@gmail.com is the designated email address, an email message such as the one below will result in a reply to the sender with a message containing the output of the submitted code.

To: icanhazoutput@gmail.com
Subject: python
Body:

```
#!/usr/bin/python
print "Hello World!"
```

The reply will contain a file, *process.log*, which has the following message:

Output produced during processing of your file
Hello World!

This is a simple example of how dryrun works. So why would this be useful to you? The usefulness of this idea is debatable. However, what it allows you to do is use any Internet-connected device to send code snippets in an email, and get back the output. As you think of ways in which you may plan to use such a service, let us next look at how you can run your service.

Running your own dryrun service

For dryrun to function, you need to make sure that the code is running to receive requests. The source code consists of three Python files:

- *dryrun_driver.py*
- *dryrun.py*
- *imaplib2.py*

dryrun_driver.py: This is the driver script for *dryrun.py*, documented separately. This script first invokes *dryrun_invoke()* to check for any existing unprocessed requests, then sets the IMAP server in IDLE mode, so as to be notified when a new email arrives. It is built on the code from <http://blog.timstooop.nl/2009/03/11/python-imap-idle-with-imaplib2/>. You will need to specify the details of the SMTP and IMAP server you plan to use, in the global variables *SMTP_SERVER*, *IMAP_SERVER*, *username* and *password*.

dryrun.py: This program implements the core logic of the program. The entry point to this program is the *dryrun_invoke*

method, which is called by the *dryrun_driver* script. In this script, the *imap_access* method is used to obtain the request details—sender, programming language and the code itself. Then, it checks if the programming language is supported, using the ‘extensions’ dictionary. If supported, the *execute_code* method is called to run it. Here, the appropriate program to process the code is chosen: *gcc* for C, *g++* for C++, and *python* or *pdflatex* for LaTeX files. The Python subprocess module is then used to run this code after it has been written to a file named *code.<extension>*, where the extension is chosen appropriately from the *extensions* dictionary. The output obtained is then stored to a file called *process.log*, which is sent back as an email to the sender, in the method *send_output*.

imaplib2.py: This is the threaded IMAP4 client based on RFC 3501 and the original *imaplib* module (<https://www.it.usyd.edu.au/~piers/python/imaplib2>). To run this service, simply execute *dryrun_driver.py*, using *python dryrun_driver.py*. Ensure the appropriate programming tools are installed, and try out how it works by sending an email in the form discussed above.

Including your language

Of course, it's very simple to extend this to include a language of your choice. As of now, you can directly modify the ‘extensions’ and ‘program’ dictionaries in the file *dryrun.py*. However, you will have to manually include the logic your particular language may need, before you get the output. For example:

- Python code execution is a one-step process
- C/C++ is a two-step process: *gcc/g++* and then *.a.out*
- LaTeX produces PDF output, so it needs to be attached separately, along with *process.log*.

Where to next?

You could use this email-based service as a ‘back-end’ service and have different front-ends, such as a bot, a Web-based interface, a smart-phone application, etc. Again, it is a project that I wrote to have fun, and I guess that's where its usefulness ends.

Let me know what you think!  amitsaha.in@gmail.com

By: Amit Saha

The author is currently working on research for a Ph.D in the area of Evolutionary Algorithms and Optimisation. As his random echoes reveal (<http://echorand.me>), he has been writing on various Linux and open source technologies over the last few years. Overall, he loves playing around with a bit of this and a bit of that. He welcomes comments on this article, and more, at amitsaha.in@gmail.com.